Guía para defender Zombis vs. Plantas

Proyecto 2 · IPC1 · Segundo semestre de 2026

Qué mostrar en el juego, qué buscar en el código y cómo explicarlo durante la evaluación en línea.

Basada en la hoja de calificación proporcionada y en la versión corregida del proyecto. Los puntos de la tabla son el valor de cada apartado en la rúbrica.

Apartado	Puntos	Página de esta guía
Documentación	10	2
Inicio, cuentas y navegación	8	2
Configuración y JSON	8	2
Catálogo de plantas	14	3
Catálogo de zombis	14	4
Motor, recursos y combate	15	5
Dificultad e IA	8	6
Hordas, podadoras y resultado	7	6
Pausa, guardado y reanudación	6	7
Estadísticas	6	7
POO, hilos y restricciones	4	8
Consola y preguntas de práctica	—	9

Antes de compartir pantalla

• Abre NetBeans, el proyecto, los dos manuales PDF y la carpeta Niveles.

• Comprueba que el juego inicia y muestra las imágenes. Usa Ctrl + F dentro de una clase para localizar métodos.

• Muestra primero el comportamiento en el juego y después el método que lo produce. Practica las explicaciones con tus propias palabras.

> “El proyecto es un juego de escritorio desarrollado en Java Swing. El jugador compra zombis con cerebros para invadir una casa defendida por plantas. El nivel se carga desde un JSON y cada cuenta conserva estadísticas y partidas guardadas.”

Documentación, cuentas y configuración

Documentación · 10 puntos

Mostrar: documentacion/Manual_tecnico.pdf y documentacion/Manual_de_usuario.pdf. El técnico vale 7 puntos: UML, arquitectura, herencia, interfaces, motor, IA, hilos y archivos. El de usuario vale 3 puntos: capturas de todo el flujo.

> “El manual técnico explica cómo está construido el programa; el manual de usuario explica cómo utilizarlo. El UML representa las clases y sus relaciones.”

Cuentas y navegación · 8 puntos

Demostración: intenta Ana! (símbolo inválido), crea Ana1, intenta repetir Ana1 y crea ana1. También puedes probar un nombre de más de 12 caracteres. Selecciona una cuenta y muestra Nueva partida, Reanudar, Estadísticas y Salir.

Clase / método	Explicación
GestorCuentas.nicknameValido()	Valida de 1 a 12 caracteres y comprueba que sean alfanuméricos.
buscarCuenta() y crearCuenta()	equals() compara el nickname exacto; se rechazan duplicados y se crean los archivos de la cuenta.
cargarCuentas()	Recorre las carpetas existentes y recupera las cuentas y sus estadísticas.
RutasCuentas.carpeta()	Codifica el nombre en hexadecimal para diferenciar mayúsculas y minúsculas también en Windows.

Cada cuenta dispone de una carpeta de partidas y un estadisticas.txt. Las cuatro clases de Archivos son RutasCuentas, GestorCuentas, LectorNivel y GestorPartidas.

Configuración y JSON · 8 puntos

Mostrar: explorador de archivos, selector Fácil/Medio/Difícil y jardines de distinto tamaño. Prueba nivel1.json (4 × 4) y nivel1_5x9.json (5 × 9) en partidas distintas.

> “LectorNivel lee el texto con java.io. Su Analizador usa un cursor para reconocer símbolos, cadenas y enteros. Valida los datos antes de construir el jardín; no utiliza una biblioteca externa de JSON.”

• **Valida:** duración positiva, cerebros no negativos, al menos una fila y una columna, filas del mismo largo e identificadores de plantas permitidos.

• **Formato esperado:** las claves van en este orden: duracionSegundos, cerebrosIniciales, jardin. Es un lector del formato específico del proyecto.

• **Código:** LectorNivel.leerNivel(), Analizador.leer(), Jardin.crearJardin() y crearPlanta(). Cada celda del JSON produce una Casilla y, si corresponde, una planta.

• **Error controlado:** en una copia del nivel sustituye una planta por PLANTA_INVALIDA y muestra que el programa rechaza el archivo.

Catálogo de plantas · 14 puntos

Mostrar: el nivel1.json original contiene las siete plantas. Compra zombis en distintos carriles para observar sus efectos. Abre Planta.java y después una clase hija.

> “Planta es una clase abstracta que reúne salud, fila, columna y operaciones comunes. Las clases concretas definen sus características y estados; MotorCombate coordina cuándo actúan.”

Planta	Datos y comportamiento que debes explicar
Lanzaguisantes	100 HP. Dispara proyectiles de 20 de daño cada 1 segundo cuando hay un blanco en su dirección.
Patatapum	50 HP. Necesita 10 segundos de preparación. Cambia de PREPARANDO a ACTIVA; causa 100 de daño a cada zombi en su casilla y explota.
Hielaguisantes	100 HP. Proyectiles de 15 de daño cada 1.5 segundos. Reduce la velocidad un 40% durante 4 segundos; los nuevos impactos renuevan el plazo sin acumular la reducción.
Nuez	400 HP. Bloquea el avance, incluso al saltador. Presentación intacta desde 66%, agrietada desde 35% y muy dañada por debajo de 35%.
Carroñívora	100 HP. Engulle zombis de su casilla o la frontal y digiere durante 8 segundos. A Zombistein le aplica 200 de daño en lugar de engullirlo. En el código se llama Carronivora.
Pinchohierba	Causa 10 de daño cada 0.5 segundos a zombis de su casilla. Los normales no la comen; Zombistein puede destruirla al aplastarla.
Bipetidora	100 HP. Dispara uno hacia el frente y dos hacia atrás cuando hay blancos en esas direcciones. Cada proyectil hace 15 de daño; cadencia de 1 segundo.

Métodos para localizar con Ctrl + F

• **MotorCombate.activarPatatapumsYDigestion():** comprueba los plazos de preparación y digestión.

• **dispararPlantas():** determina si hay blancos y crea los proyectiles respetando la cadencia.

• **pinchohierbaYPatatapum():** aplica daño por casilla y resuelve también la Carroñívora.

• **PanelJardinFondo.dibujarEstadoPlanta():** representa grietas de la Nuez, preparación de Patatapum y digestión.

Ejemplo para responder

> “La cadencia indica cuánto se espera entre ataques, no el daño por segundo directamente. Hielaguisantes hace 15 de daño por impacto cada 1.5 segundos, y además modifica temporalmente la velocidad del zombi.”

Catálogo de zombis · 14 puntos

Mostrar: el catálogo de compra, sus costos y algunas diferencias de movimiento, ataque y apariencia. Zombidito y los bailarines secundarios aparecen por otras entidades.

> “Zombi es una clase abstracta con salud, daño, costo, velocidad, fila, posición y estado de ataque. Sus clases hijas agregan escudos, salto, baile o lanzamiento.”

Zombi	Datos y comportamiento que debes explicar
Común	100 HP; costo 50; velocidad ×1; daño 15 por segundo. Se detiene a comer y pierde el brazo al llegar al 50% de salud.
Caracono	100 HP y 100 de escudo; costo 100; velocidad ×1. Primero recibe daño el cono. Su ataque es 20 con cono y 15 después de perderlo.
Garrocha	120 HP; costo 100; velocidad ×1.7; daño 15 por segundo. Puede superar la primera planta salvo Nuez. Tras usar la garrocha queda a velocidad ×1.
Deportista	100 HP y 150 de escudo; costo 150; velocidad ×2. Causa 10 de daño cada 0.5 segundos. El casco absorbe primero el daño.
Bailón	150 HP; costo 150; velocidad ×0.8; daño 15 por segundo. Cada 5 segundos baila durante 1 segundo e intenta generar bailarines en las cuatro casillas adyacentes disponibles.
Zombistein	300 HP; costo 350; velocidad ×0.7. Aplasta plantas. Al llegar a la mitad de salud lanza una vez a Zombidito antes de la penúltima casilla.
Zombidito	50 HP; velocidad ×1.7. Aparece únicamente por el lanzamiento; no se compra ni se cobra al jugador.

Polimorfismo con un ejemplo real

Abrir: ZombiCaracono.recibirDanio() y, si lo piden, ZombiDeportista.recibirDanio().

> “Si al cono le quedan 30 puntos y recibe 50 de daño, se destruye el escudo y los 20 restantes pasan a la salud. El motor llama a recibirDanio(), pero la clase concreta decide cómo repartirlo. Eso demuestra polimorfismo.”

Métodos del motor relacionados

- **moverYAtacarZombis():** movimiento, obstáculos, salto, baile y lanzamiento.
- **atacarConMordisco():** aplica daño cuando vence el plazo del siguiente ataque.
- **invocarBailarines():** revisa límites y disponibilidad de casillas adyacentes.
- **PanelJardinFondo.spriteZombi():** selecciona imágenes según el tipo y su estado.

Motor, recursos y combate · 15 puntos

Recorrido de demostración

1. Señala tiempo y cerebros. 2. Selecciona un zombi y pulsa un carril. 3. Muestra el descuento. 4. Intenta una compra sin fondos. 5. Recoge un cerebro. 6. Muestra disparos, daño y eliminación gráfica.

Invocación y control

Abrir: Partida.invocarZombi() y VentanaPartida.invocar().

> “Se comprueba que la partida permita jugar, que el carril exista y que alcancen los cerebros. Una compra válida registra el zombi, descuenta el costo y actualiza los contadores. Los zombis generados por otros zombis se registran por otra vía y no se cobran.”

Paso general del motor

Abrir: MotorCombate.actualizar(double segundos).

Orden principal: actualizar economía → defensa automática → estados temporales → disparos → proyectiles → movimiento y ataques → daño y efectos por casilla.

> “El motor recibe el tiempo transcurrido entre actualizaciones. La posición es continua para obtener un movimiento fluido. El reloj visible y la simulación utilizan el mismo paso de tiempo limitado.”

Economía y cerebros

Regla	Implementación / explicación
Generadores	Fácil: 6; Medio: 5; Difícil: 4. Cada uno usa su propio plazo aleatorio de 5 a 15 segundos.
Duración y recolección	Cada icono permanece 5 segundos y entrega 25 cerebros al pulsarlo. recogerCerebro() verifica que siga vigente y la partida esté activa.
Planta destruida	destruirPlanta() retira la planta, suma el contador y crea entre 3 y 6 cerebros.
Métodos	actualizarGenerador(), recogerCerebro(), actualizarEconomia() y destruirPlanta().

Colisión del proyectil

Abrir: moverProyectiles(). Se revisa el tramo entre la posición anterior y la nueva. Entre blancos del mismo carril se elige el primero alcanzado. Se aplica el daño o efecto y se retira el proyectil.

Casilla lógica

```
int c = (int) Math.floor(z.getPosicionX() + 0.35);
```

Abrir: casillaActual(). El centro del zombi determina una columna única, limitada a los bordes del jardín. Esto permite resolver Patatapum, Pinchohierba, Carroñívora, invocaciones y otras reglas espaciales aunque el movimiento sea continuo.

Dificultad, hordas y final de partida

Dificultades e IA · 8 puntos

Abrir: MotorCombate.actualizarDefensa(). Fácil no siembra; Medio elige una planta al azar cada 10 segundos y busca una casilla libre desde el lado más cercano a la casa. Difícil evalúa cada 5 segundos.

Prioridades en Difícil: 1) amenaza cerca de casa; 2) zombi que rebasó una planta; 3) resistente; 4) deportista; 5) saltador; 6) más de tres zombis en un carril; 7) Lanzaguisantes en el carril menos defendido con espacio, si ninguna regla logró actuar.

> “Se revisa una prioridad en todo el tablero antes de pasar a la siguiente. Cuando una siembra tiene éxito, return termina la evaluación para ejecutar una única acción.”

Hordas, podadoras y resultado · 7 puntos

Abrir: Partida.getBloque(), costoActual(), MotorCombate.llegadaALaCasa() y VentanaPartida.finalizar().

• La duración se divide en 7 bloques iguales. En el bloque 4 se paga 50% del costo; en el bloque 7 se paga 25%. Fuera de esos periodos se usa el precio normal.

• El común cuesta 50 normalmente, 25 en bloque 4 y 13 en bloque 7: los valores fraccionarios se redondean hacia arriba.

• La primera llegada consume la podadora y elimina todos los zombis del carril. Una llegada posterior en ese mismo carril produce la victoria.

• Al llegar el tiempo a cero sin invasión hay derrota. Se detiene la partida, se registran estadísticas y se muestra el resumen.

Nivel corto opcional para demostrarlo

Crea un archivo separado llamado defensa_70s.json. Respeta el orden de sus claves y ejecútalo en Fácil:

```
{
  "duracionSegundos": 70,
  "cerebrosIniciales": 500,
  "jardin": [
    ["VACIO", "VACIO", "VACIO", "VACIO"]
  ]
}
```

Partida A: no compres zombis. El bloque 4 comienza a los 30 segundos transcurridos; el bloque 7, a los 60; la derrota, a los 70. Señala el cambio de precios y la notificación.

Partida B: compra un común, espera a que active la podadora y después compra otro en el mismo carril para mostrar la victoria. Usa los niveles originales para demostrar plantas.

Pausa, archivos y estadísticas

Pausa, guardado y reanudación · 6 puntos

Demostración: compra un zombi y espera movimiento o disparos → pausa → señala reloj y entidades detenidas → Guardar y salir → Reanudar partida guardada → selecciona el archivo → muestra que carga pausada → pulsa Reanudar.

Abrir: VentanaPartida.abrirPausa(), guardarYSalir(); GestorPartidas.guardar(), cargar() y listar().

> “El archivo binario contiene el estado del modelo mediante ObjectOutputStream: motor, partida, entidades, posiciones, salud y tiempos internos. Al cargar se recupera ese estado y se construye de nuevo la ventana. No se serializan las ventanas ni los hilos.”

• Solo se guarda una partida pausada y activa. El archivo se identifica con fecha y hora y se coloca en la carpeta de partidas de la cuenta.

• El guardado usa primero un archivo temporal y después lo renombra. Al recuperar se verifica la cuenta y se deja la partida pausada.

• La fracción de segundo también se guarda para continuar el reloj con su estado previo.

Estadísticas · 6 puntos

Mostrar: después de finalizar una partida, abre Estadísticas y sus pestañas Mi cuenta y Ranking global. Abre también el estadisticas.txt de la cuenta para mostrar el formato legible.

Abrir: Estadisticas.java; GestorCuentas.escribirEstadisticas() y cargarEstadisticas(); VentanaMenuPrincipal.estadisticas() y ranking().

Dato o cálculo	Qué explicar
Contadores	Partidas jugadas, ganadas y perdidas; plantas destruidas; zombis comprados; cerebros gastados; podadoras activadas y mejor tiempo de victoria.
Porcentaje	Ganadas × 100.0 / jugadas. Ejemplo: 3 victorias en 4 partidas = 75%.
Promedio	Plantas destruidas / partidas jugadas. Ejemplo: 20 plantas en 4 partidas = 5. Se evita dividir entre cero.
Ranking	Hasta 5 cuentas. Rapidez: menor tiempo primero y solo cuentas con victorias. Destrucción y éxito: mayor cantidad primero.
Ordenamiento	Inserción sobre arreglos nativos. Los empates se resuelven por nickname.

Detalle: bailarines y Zombidito generados no cuentan como compras. La bandera resultadoRegistrado en finalizar() evita registrar dos veces el mismo resultado.

POO, hilos y restricciones · 4 puntos

Paquete	Responsabilidad
Principal	Arrancar la aplicación. main() usa SwingUtilities.invokeLater().
Vista	Ventanas, dibujo, eventos y coordinación de la presentación.
Modelo	Estado y reglas de la partida.
Plantas y zombis	Entidades y características particulares.
Archivos	Lectura de niveles y persistencia de cuentas y partidas.

Ejemplos que puedes señalar en el código

• **Encapsulamiento:** atributos privados y métodos para consultar o cambiar el estado.

• **Herencia:** Lanzaguisantes extends Planta; ZombiCaracono extends Zombi.

• **Polimorfismo:** recibirDanio() redefine la aplicación de daño en clases con escudo.

• **Interfaces:** Serializable permite serializar el modelo; SeleccionCarril comunica una fila seleccionada desde el tablero.

• **Arreglos:** cuando se llenan se crea uno más grande y se copian los elementos. No se utiliza Collections Framework para almacenar las entidades.

Hilos: iniciar, publicar y detener

Abrir: HilosPartida.iniciar(), trabajador() y detener().

> “Hay un hilo de reloj y un hilo por generador. Los trabajadores esperan y publican tareas mediante invokeLater. La modificación del modelo y de la interfaz se realiza en el hilo de eventos de Swing, el EDT.”

Dos tiempos diferentes: los trabajadores despiertan cada 40 milisegundos para solicitar actualizaciones; los cerebros aparecen cuando se cumple su plazo aleatorio de 5 a 15 segundos.

> “Al pausar se interrumpen los trabajadores y cambia el número de sesión. Las tareas pendientes de una sesión antigua se descartan. Al reanudar se crean trabajadores nuevos conservando el estado del juego.”

Por qué esto ayuda: los trabajadores no leen ni modifican directamente el modelo compartido. Las operaciones del modelo se ejecutan por turnos en el EDT, sin añadir bloqueos manuales en el código del juego.

Restricciones técnicas que debes reconocer

Aplicación con Java estándar y Swing; lectura propia de JSON; arreglos nativos. Sin dependencias externas para ejecutar el juego, Lombok, variables globales public static ni mecanismos manuales de sincronización prohibidos. Las herramientas usadas para producir documentación son distintas de las dependencias de ejecución del juego.

Consola y preguntas de práctica

Qué se ve en la salida de NetBeans

El juego muestra su estado en ventanas Swing. La consola informa de compilación, ejecución y algunos errores de archivos; no imprime cada ataque. Que permanezca sin mensajes durante el combate puede ser normal.

Según el sistema de construcción, al terminar correctamente puede aparecer BUILD SUCCESS o BUILD SUCCESSFUL. Eso confirma ese proceso; la funcionalidad del juego se demuestra en la interfaz.

Para abrir el ejecutable desde la carpeta del proyecto en Windows, con Java 21 o superior instalado:

```
java -jar dist\ZombisVsPlantas.jar
```

El JAR incluye las clases y los recursos gráficos. Para compilar desde NetBeans, utiliza Clean and Build sobre el proyecto; para iniciar desde el código, utiliza su clase principal.

Salida de las pruebas revisadas

```
OK: 52 comprobaciones.
OK: 32 comprobaciones de concurrencia y reanudación.
OK: 22 comprobaciones visuales y de lanzamiento.
```

Estas líneas pertenecen a Pruebas, PruebasHilos y PruebasVisuales. Aparecen al ejecutar las pruebas, no automáticamente al abrir el juego. Son comprobaciones concretas y no sustituyen la demostración de toda la rúbrica.

Preguntas frecuentes

Pregunta	Respuesta que debes comprender
¿Qué es dt?	Tiempo transcurrido entre actualizaciones. Permite mover entidades según el tiempo y no por una cantidad fija por fotograma.
¿Por qué limitarlo a 0.15 s?	Evita un salto grande de simulación si la ventana se retrasa. El reloj y el motor usan el mismo paso limitado.
¿Cómo ralentiza el hielo?	Multiplica el avance por 0.60 durante el efecto. Otro impacto renueva el vencimiento.
¿Qué hace super?	Invoca el constructor o una implementación de la clase padre.
¿Qué indica @Override?	Que el método redefine uno heredado; el compilador comprueba esa relación.
¿Por qué una clase abstracta?	Reúne características comunes sin crear directamente una planta o zombi genérico.
¿Por qué guardar todo el modelo?	Para recuperar también posiciones, salud, recursos, efectos y tiempos internos.

Recorrido final para presentar: inicio → cuentas → JSON → combate → pausa y guardado → recuperación → resultado → estadísticas → código y manuales. Explica con tus palabras y señala el método que produce cada comportamiento.